Technology-Aware Logic Synthesis

OU, XIN-HONG
National Taiwan University
r14921060@ntu.edu.tw

LI, YUN-RU
National Taiwan University
r14943065@ntu.edu.tw

Abs@ract

Keywords—keyword1, keyword2, keyword3, keyword4, keyword5

1 Introduction

This section introduces the background, motivation, and contributions of your research.

1.1 Background

Logic synthesis is a fundamental step in digital design automation, significantly impacting the Quality of Results (QoR) such as area, timing, and power consumption. Traditional logic synthesis frameworks, such as ABC [3], employ greedy local optimization approaches that apply incremental changes to small portions of the circuit. These methods, while effective, often limit the exploration space and may miss opportunities for significant improvements.

Recently, E-Syn [1] has emerged as a novel logic optimization framework that leverages equivalence graphs (e-graphs) for Boolean logic rewriting. E-Syn addresses the limitations of traditional AIG-based synthesis by maintaining equivalent classes of logic specifications, allowing exploration of a wider range of equivalent logic forms. A key innovation of E-Syn is its use of technology-aware cost functions obtained from machine learning models, specifically XGBoost regression, to bridge the gap between technology-independent logic cost and post-mapping QoR. The overall workflow of the E-Syn framework is illustrated in Figure 1.

1.2 Motivation

In E-Syn, the extraction phase plays a critical role in selecting the optimal logic form from the e-graph based on cost estimates. The quality of these cost estimates directly impacts the final QoR of the synthesized circuit. The current E-Syn framework uses XGBoost regression models to predict post-mapping area and delay from technology-independent features. However, the accuracy of these regression models has a direct influence on the extraction quality. Estimation errors in the cost function can lead to suboptimal candidate selection during extraction, potentially missing better logic forms that would result in superior QoR.

To address this limitation, we propose to optimize the regression training process in E-Syn to reduce estimation errors. By improving the accuracy of the cost prediction models, we can enhance the quality of extraction decisions, leading to better final synthesis results. Specifically, we focus on:

- Improving the training data quality and feature engineering for the regression models
- Optimizing hyperparameters and model architectures to reduce prediction errors
- Enhancing the cost function accuracy to better reflect actual post-mapping QoR
- Validating that improved regression accuracy translates to better extraction quality and final synthesis results

1.3 Contributions

The main contributions of this work include:

- An analysis of the impact of regression model accuracy on E-Syn's extraction quality and final synthesis results
- Investigation of how different features affect prediction errors in cost estimation models, identifying critical features that significantly influence estimation accuracy
- Comprehensive evaluation demonstrating that reduced estimation errors lead to better extraction decisions and improved QoR
- Comparison with multiple regression models (XGBoost [4], Random Forest [5], LightGBM [6], MLP [7], CatBoost [8]) to identify the most suitable approach for cost estimation

1.4 Paper Organization

The rest of this paper is organized as follows: Section 2 formulates the problem of regression model optimization for E-Syn's cost estimation. Section 3 presents our proposed methodology for improving regression training. Section 4 presents the experimental results and discussion. Finally, Section 5 concludes the paper.

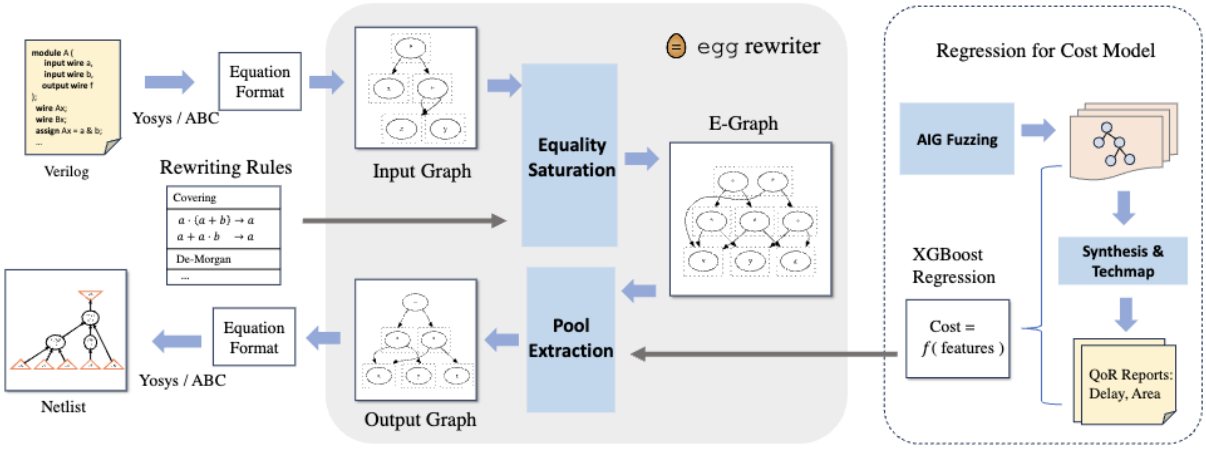

Figure 1: Overview of the E-Syn framework workflow. The framework uses e-graph rewriting to explore equivalent logic forms and employs machine learning-based cost functions to guide the extraction phase for optimal synthesis results.

2 Problem Formulation

As discussed in the introduction, the accuracy of regression models used for cost estimation in E-Syn directly impacts the quality of extraction decisions and the final synthesis results. The original E-Syn implementation primarily uses XGBoost for cost estimation and relies on basic features extracted from the circuit representation, including AST size (ASTSize), AST depth (ASTDepth), operator counts (+, !, *, &), Liberty cost metrics (SUM_LIB, AVE_LIB), and total node count (SUM_NODE). While these features provide fundamental information about the circuit structure, they may not fully capture the complexity and characteristics that influence post-mapping area and delay.

To improve the overall performance of E-Syn, we focus on optimizing the regression training process from two perspectives:

1. **Modifying training models and methodologies**: We investigate different regression models and training approaches to reduce prediction errors. This includes exploring various machine learning algorithms beyond XGBoost and optimizing their hyperparameters, training procedures, and model architectures to achieve better prediction accuracy for post-mapping area and delay estimation.

2. **Identifying useful features for prediction**: We analyze the impact of different features on prediction errors and identify which features are most critical for accurate cost estimation. This involves extending the feature set beyond the basic features currently used in E-Syn, performing feature importance analysis, and understanding how different circuit characteristics contribute to the prediction accuracy of area and delay.

By addressing these two aspects, we aim to develop more accurate cost estimation models that can better guide the extraction phase in E-Syn, ultimately leading to improved Quality of Results (QoR) in logic synthesis.

3 Proposed Method

Our approach focuses on two main aspects: exploring different regression models and enhancing feature engineering for cost estimation.

3.1 Regression Models

The original E-Syn framework primarily uses XGBoost for cost estimation. To improve prediction accuracy, we investigate multiple regression models and evaluate their performance. We select the following models based on their distinct characteristics and potential advantages:

- **XGBoost**: As the baseline model used in E-Syn, XGBoost is a gradient boosting framework that excels at handling non-linear relationships and feature interactions. It provides excellent performance for structured data and offers built-in regularization to prevent overfitting.

- **Random Forest**: We include Random Forest as an ensemble learning method that combines multiple decision trees. It is robust to overfitting and can capture complex interactions between features through its bagging approach, providing a different perspective compared to gradient boosting methods.

- **LightGBM**: LightGBM is another gradient boosting framework that uses a leaf-wise tree growth strategy, making it faster and more memory-efficient than traditional gradient boosting methods. It is particularly suitable for large-scale

datasets and can handle high-dimensional feature spaces effectively.

- **Multi-Layer Perceptron (MLP)**: We incorporate MLP, a deep neural network model, to capture complex non-linear relationships that may not be easily modeled by tree-based methods. MLP can learn hierarchical feature representations and is capable of modeling intricate patterns in the data through its multiple hidden layers and non-linear activation functions. This makes it particularly valuable for discovering non-obvious relationships between circuit characteristics and post-mapping QoR.

- **CatBoost**: CatBoost is a gradient boosting algorithm designed to handle categorical features effectively and reduce overfitting through its ordered boosting technique. It provides robust performance with minimal hyperparameter tuning, making it a practical choice for cost estimation.

By comparing these diverse models, we aim to identify the most suitable approach for cost estimation in logic synthesis and potentially combine their strengths through ensemble methods.

3.2 Feature Engineering

To enhance the predictive power of our regression models, we extend the feature set by incorporating additional feature categories beyond the basic features used in the original E-Syn implementation. In total, we add **27 new features** across four categories. The complete list of these 27 features and their descriptions are presented in Table 1.

3.3 Training Methodology

For each regression model, we employ a systematic training approach. The training process consists of the following steps:

1. **Hyperparameter Optimization**: For tree-based models (XGBoost, LightGBM, CatBoost, Random Forest), we perform grid search with 10-fold cross-validation to identify optimal hyperparameters. The hyperparameter search space is designed based on the characteristics of each model type, balancing model complexity and generalization ability. For MLP, we employ a manual hyperparameter search approach, testing different combinations of hidden layer dimensions, learning rates, and dropout rates on the validation set.

2. **Model Training**: After identifying the best hyperparameters, we train the final model on the training set. For tree-based models, we use the optimized hyperparameters directly. For MLP, we employ early stopping based on validation loss to prevent overfitting.

3. **Feature Scaling**: We apply standard scaling to all features before training, ensuring that features with different scales contribute equally to the model learning process. This is particularly important for neural network models like MLP.

4. **Model Evaluation**: We evaluate each trained model on a separate validation set using Mean Absolute Percentage Error (MAPE) to assess prediction accuracy.

This systematic approach ensures that all models are trained and evaluated under consistent conditions, enabling fair comparison of their performance for cost estimation in logic synthesis.

4 Experimental Results and Discussion

4.1 Experimental Setup

4.1.1 Dataset

The dataset is collected from circuit analysis using the E-Syn framework. Circuit data is extracted from S-expression files, and features are computed to represent various circuit characteristics. The dataset consists of 40,000 training samples, 10,000 validation samples, and 5,000 test samples. The training and validation sets are used for model training and hyperparameter tuning, while the test set is reserved for final model evaluation.

4.1.2 Implementation Details

Regression model training and evaluation are implemented in Python using scikit-learn, XGBoost, LightGBM, CatBoost, and PyTorch libraries. Our implementation is based on the open-source E-Syn framework [2]. All experiments are performed on a Ubuntu 22.04.3 LTS server equipped with Intel Core i9-13900K processors, an NVIDIA GeForce RTX 4070 Ti GPU, and 125GB memory.

4.2 Regression Model Results

4.2.1 Comparison of Different Models

We compare different regression models (XGBoost, Random Forest, LightGBM, MLP, and CatBoost) using Mean Absolute Percentage Error (MAPE). Figure 2 shows the results. XGBoost achieves the lowest MAPE (99.46%), significantly outperforming other models (MLP: 343.63%, CatBoost: 445.68%, LightGBM: 485.17%, Random Forest: 499.56%).

XGBoost's superior performance can be attributed to its gradient boosting framework with built-in regularization, which effectively handles non-linear relationships and feature interactions in circuit cost estimation while preventing overfitting. The model's tree-based structure captures complex decision boundaries in the mapping from technology-independent features to post-mapping area.

Table 1: New Features Added to E-Syn Cost Estimation Models

Category	Feature Name	Description
Structural Features (5)	num_internal_nodes	Number of internal nodes (non-leaf nodes) in the circuit, representing the complexity of the logic structure
	avg_fanin	Average number of input connections per node, indicating the average input dependency
	max_fanin	Maximum number of input connections for any single node, identifying highly connected nodes
	avg_fanout	Average number of output connections per node, showing how signals are distributed
	max_fanout	Maximum number of output connections for any single node, identifying nodes that drive many other nodes
Syntactic Features (11)	count_and	Total count of AND operators (& and *) in the circuit
	count_or	Total count of OR operators (+) in the circuit
	count_xor	Total count of XOR operators (^) in the circuit
	count_not	Total count of NOT operators (!) in the circuit
	total_operators	Sum of all operator counts, representing the total number of logic operations
	ratio_and	Proportion of AND operators relative to total operators
	ratio_or	Proportion of OR operators relative to total operators
	ratio_xor	Proportion of XOR operators relative to total operators
	ratio_not	Proportion of NOT operators relative to total operators
	num_parentheses	Number of parentheses pairs in the expression, indicating structural nesting
	max_nesting_depth	Maximum depth of nested expressions, measuring the deepest level of logic hierarchy
Expression Complexity Features (7)	avg_expr_length	Average length (number of nodes) of sub-expressions, measuring typical expression size
	max_expr_length	Maximum length of any sub-expression, identifying the largest expression component
	min_expr_length	Minimum length of any sub-expression, identifying the smallest expression component
	std_expr_length	Standard deviation of expression lengths, measuring variability in expression sizes
	avg_nesting_depth	Average nesting depth across all nodes, indicating typical hierarchy level
	min_nesting_depth	Minimum nesting depth, showing the shallowest hierarchy level
	std_nesting_depth	Standard deviation of nesting depths, measuring variability in hierarchy structure
Graph Features (4)	graph_nodes	Total number of nodes in the circuit graph representation
	graph_edges	Total number of edges (connections) in the circuit graph
	graph_density	Ratio of actual edges to maximum possible edges, measuring graph connectivity density
	avg_degree	Average degree (number of connections) per node in the graph

Other models exhibit higher MAPE values due to various limitations. MLP struggles with the relatively small training dataset size, as deep neural networks typically require large amounts of data. Random Forest and LightGBM show higher errors, potentially due to their different boosting strategies that may not be as well-suited for this specific task. CatBoost performs better than Random Forest and LightGBM but still falls short of XGBoost.

Given these results, XGBoost is the most suitable model among the five evaluated approaches for cost estimation in logic synthesis.

4.2.2 Comparison of Different Feature Sets

We compare models trained with only the original E-Syn features against models trained with new features (including the 27 additional features). Figure 3 shows the MAPE comparison results. The model using only original features achieves lower MAPE compared to the model using new features, indicating that the original feature set is more effective for cost estimation in this context.

The inferior performance of the new features can be attributed to two main factors. First, the circuit generation program used to create training data was implemented separately to generate circuits containing XOR gates. This custom implementation results in circuits that differ significantly from other circuits in the dataset, creating a distribution mismatch that affects the model's ability to generalize from the new features. Second, E-Syn stores circuits in S-expression (sexpr) format, while training data generation uses equation

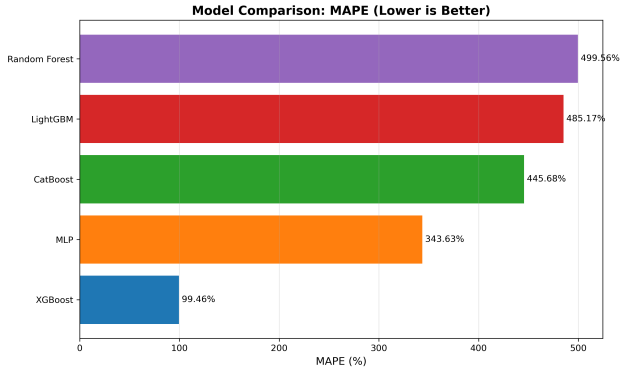

Figure 2: MAPE comparison across different regression models for area estimation.

(eqn) format. This format discrepancy may introduce inconsistencies in feature extraction between the two representations, leading to misalignment in the feature values computed from different circuit formats.

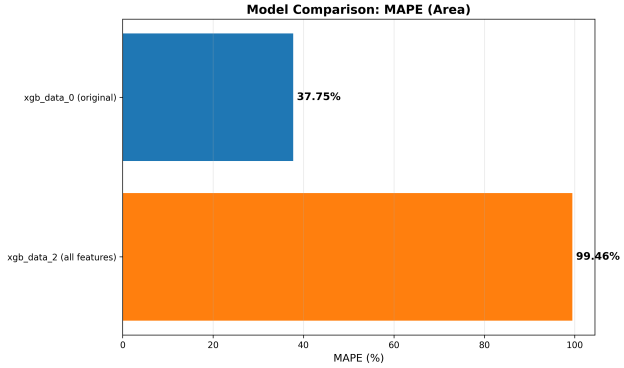

Figure 3: MAPE comparison between models trained with original features only and new features for area estimation.

5 Conclusion

References

[1] Chen Chen, Guangyu Hu, Dongsheng Zuo, Cunxi Yu, Yuzhe Ma, and Hongce Zhang, "E-Syn: E-Graph Rewriting with Technology-Aware Cost Functions for Logic Synthesis," 2024.

[2] Guangyu Hu, "E-Syn: E-Graph Rewriting with Technology-Aware Cost Functions for Logic Synthesis," GitHub repository, 2024. [Online]. Available: `https://github.com/Gy-Hu/E-Syn`

[3] Robert Brayton and Alan Mishchenko, "ABC: An Academic Industrial-Strength Verification Tool," *Computer Aided Verification*, Springer, 2010, pp. 24–40.

[4] J. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, San Francisco, CA, USA, 2016, pp. 785–794.

[5] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001.

[6] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 3146–3154.

[7] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, Cambridge, MA, USA: MIT Press, 2016.

[8] L. Prokhorenkova, G. Gusev, A. Vorobev, A. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018, pp. 6638–6648.